

This API architecture follows RESTful principles, focusing on scalability, security, and developer experience.

1. API Overview

- **Base URL:** `https://api.fitness-coach.ai/v1`
- **Protocol:** HTTPS
- **Content Type:** `application/json`
- **Versioning:** URI-based (`/v1/`)
- **Rate Limiting:** 1000 requests/day per user (burst up to 50).

2. Authentication

- **Mechanism:** OAuth 2.0 / OpenID Connect with JWT (JSON Web Tokens).
- **Header:** `Authorization: Bearer `
- **Storage:** Tokens should be stored in an `httpOnly` secure cookie for web clients.

3. Key Endpoints

Method	Endpoint	Description
POST	/auth/register	Create a new user profile.
POST	/auth/login	Authenticate and return JWT.
GET	/users/me	Fetch profile and equipment list.
POST	/plans/generate	Trigger AI to create a new workout plan.
GET	/workouts/today	Fetch the current day's scheduled workout.
POST	/workouts/{id}/logs	Log exercise performance.
POST	/measurements	Update weight/body stats (limit 1/day).

4. Request/Response Examples

Generate Workout Plan

Endpoint: `POST /plans/generate`

Request Body:

```
{
  "focus": "hypertrophy",
  "intensity": "high"
}
```

Success Response (201 Created):

```
{
  "plan_id": "uuid-12345",
  "status": "generated",
  "summary": "4-day split focus on strength."
}
```

Log Workout

Endpoint: `POST /workouts/{id}/logs`

Request Body:

```
{
  "exercise_name": "Bench Press",
  "sets": 3,
  "reps": 10,
  "duration_minutes": 20
}
```

5. Standard Error Responses

The API uses standard HTTP status codes to indicate success or failure.

- ****400 Bad Request:**** Missing fields or schema validation failure.
- ****401 Unauthorized:**** Invalid or expired JWT.
- ****403 Forbidden:**** User attempting to access another user's private data.
- ****429 Too Many Requests:**** Rate limit exceeded.
- ****422 Unprocessable Entity:**** Business rule violation (e.g., trying to log weight twice in one day).

Example Error Body:

```
{
  "error": {
    "code": "MEASUREMENT_ALREADY_EXISTS",
    "message": "You can only update your measurements once per 24 hours.",
    "request_id": "req-998877"
  }
}
```

6. Validation Logic (Middleware/Service Layer)

1. **Authentication Guard:** Middleware checks `Authorization` header against the JWT signature.

2. **Input Sanitation:**

* `age`: Validate `int` ≥ 18 .

* `weight\_kg`: Validate `float` > 0 .

* `exercise\_name`: Sanitize strings to prevent NoSQL/SQL injection (if using raw queries).

3. **Business Logic Guard:**

* `POST /measurements`: Check `User\_Measurements` table for an existing record with
`CAST(recorded\_at AS DATE) = CURRENT\_DATE` for this user before allowing insertion.

4. **AI Latency Handling:**

* The `POST /plans/generate` endpoint returns `202 Accepted` if the AI generation is queued for processing, with a `poll\_url` provided in the header/body.

7. Security Headers

To satisfy GDPR/CCPA and security requirements:

- `Strict-Transport-Security` (HSTS)
- `X-Content-Type-Options: nosniff`
- `Content-Security-Policy` (CSP)
- `Cache-Control: no-store` (prevents PII from being cached in proxies/browsers)